

BIG Certificate Portal — Developer Report

Blockchain Certificate Issuance & Verification System

Report generated: 2026-09-30 · Origin: /home/pradeep/certificate-app · Repository: (none — direct filesystem deploy)

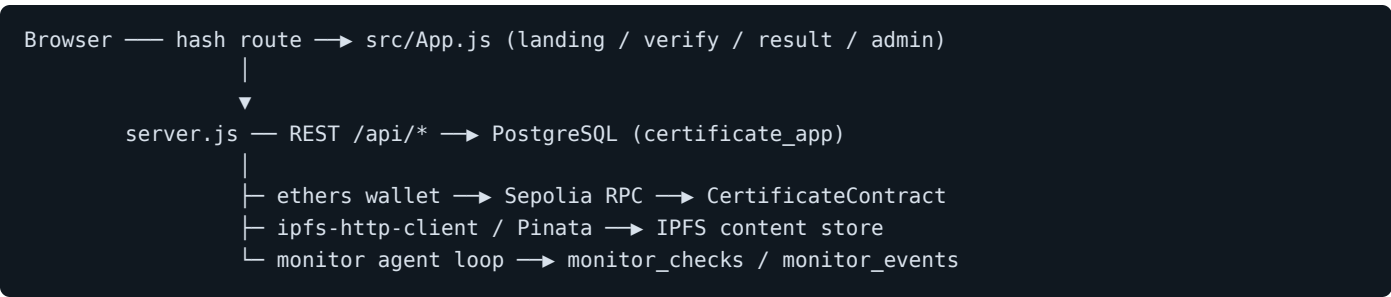
1. Overview

Full-stack web platform that issues, stores, and verifies student certificates. Each certificate is hashed (`sha256`), stored via IPFS/Pinata, and its fingerprint is anchored to an Ethereum smart contract (Sepolia testnet) through an `ethers` wallet. A rich admin portal manages certificates, results, issues, campus app monitoring, and a data-privacy module (DPIA / ROPA workflows). Public users verify certificates by hash and retrieve results via SMS OTP.

2. Technology Stack

Layer	Technology
Backend API	Node.js + Express 5 (<code>server.js</code> , 1970 lines)
Frontend	React 19 via Create React App 5 (<code>src/</code>), hash-based routing (no router library)
Database	PostgreSQL 16 via <code>pg</code> pool; <code>db.js</code> auto-creates the DB + schema + idempotent migrations on boot
Blockchain	<code>ethers</code> v6, JSON-RPC on Sepolia, inline contract ABI
Content Store	<code>ipfs-http-client</code> + Pinata JWT (<code>PINATA_JWT</code>)
File Handling	<code>multer</code> (in-memory), <code>xlsx</code> (bulk uploads / exports)
Auth	SHA-256 password verification (legacy + current), header-based admin tokens
Other	<code>cors</code> , <code>qrcode.react</code> , <code>axios</code> , <code>form-data</code>
Testing	Jest backend suite (<code>tests/server.test.js</code>) + CRA test files

3. Architecture Flow



Deployment: Express serves the production React build (`build/`) on `:5000` ; the CRA dev server runs on `:3000` . Two LAN hosts: `192.168.29.71` and `192.168.29.85` (the latter supervised by PM2 under app name `cert-app`).

4. API Endpoints (Express)

4.1 Public / Session

- `POST /api/admin/login` — admin authentication
- `GET /api/verify/:hash` — on-chain + DB authenticity check
- `POST /api/certificates`, `GET /api/certificates/:hash`, `/api/certificates/download`, `/api/certificates/blockchain`
- `/api/certificates/bulk-upload`, `/api/certificates/bulk-save`
- `GET /api/search`; `/api/departments`, `/api/departments/:name`
- `POST /api/results`, `GET /api/results/:id`, `/api/results/verify`
- `/api/results/otp/request`, `/api/results/otp/verify` (in-memory OTP store with TTL + attempt limits)
- `/api/results/bulk-upload`, `/api/results/bulk-save`

4.2 Authenticated (`requireAdmin` / `requirePerm(perm)`)

- `GET /api/dashboard/stats`, `/api/dashboard/analytics`
- `/api/activity`, `/api/brand`, `/api/issues`, `/api/issues/:id`
- `/api/users`, `/api/users/:username`, `/api/settings/sms`, `/api/settings/sms/test`
- `/api/monitor/apps`, `/api/monitor/apps/:id`, `/api/monitor/apps/:id/checks`, `/api/monitor/stats`
- `POST /api/agent/event` — monitor agent ingest (token-guarded)

4.3 DPIA / Privacy Module

- `GET /api/dpia` (list), `POST /api/dpia`, `GET|PUT|DELETE /api/dpia/:id`
- `POST /api/dpia/:id/risk`, `PUT /api/dpia/risk/:riskId`
- `GET /api/dpia/dashboard` — aggregate KPIs & charts
- `GET /api/dpia/report` — executive text report (must be declared **before** `/api/dpia/:id`)
- `GET|PUT /api/dpia/workflow` — stage/transition config
- `POST /api/dpia/:id/transition` — workflow state transition + audit log

5. Database Schema (`certificate_app`)

All tables created with `CREATE TABLE IF NOT EXISTS` followed by idempotent `ALTER TABLE ... ADD COLUMN IF NOT EXISTS` and index rules:

Table	Purpose
certificates	Issued certificates (hash, ipfsHash, txHash, student/course details)
results	Semester results (+ aadhaar, mobile)
users	Accounts + permissions (JSONB) + departments
departments, brand, issues, activity	Affiliated data, branding, issue tracking, activity log
app_sessions, app_settings	Session + system settings
monitor_apps	Monitored campus apps (+ ROPA / DPIA / DPDP status fields)
monitor_checks, monitor_events	Uptime checks and events
dpia_assessments	DPIAs (JSONB data_categories/risks, third_party, stage, risk_level)

dpia_risks	Risks per DPIA (FK cascade delete)
dpia_workflow_config	Workflow stages/transitions (JSONB)
dpia_audit_log	Stage-change audit trail

6. Frontend Structure (`src/`)

File	Role
<code>App.js</code>	Hash router + landing page (verify, result, admin)
<code>AdminPage.js</code> (3681 lines)	13 modules: Dashboard, Campus Monitor, Issue Certificate, Bulk Upload, Verify, Certificates, Results, Analytics, Issues, DPIA, Activity, Users, Settings, Downloads
<code>VerifyPage.js</code> / <code>VerifyResult.js</code>	Public certificate verification / result display with OTP
<code>searchPage.js</code>	Certificate search
<code>config.js</code>	Brand constants (university name, SVG logo)
<code>certificate-blockchain/</code>	Blockchain client helpers

7. Configuration (`.env`)

```
ADMIN_USERNAME= admin          ADMIN_PASSWORD= admin123
ADMIN_PRIVATE_KEY=             SEPOLIA_RPC_URL=
CONTRACT_ADDRESS=             PINATA_JWT=
DB_HOST/DB_PORT/DB_USER/DB_PASS/DB_NAME    (postgres / certificate_app)
MONITOR_AGENT_TOKEN / MONITOR_APP_KEY / MONITOR_POLL_INTERVAL / MONITOR_CHECK_TIMEOUT
```

8. Recent Fixes & Known-Good State

- **Report route shadowing:** `/api/dpia/report` moved above `/api/dpia/:id` — resolved the 500 `bigint` cast error (`"report"` was being parsed as an id).
- **CamelCase aliases quoted** (`AS "highRisk"` , `"openRisks"` , `"overdue"` , `"thirdParty"`) — Postgres lowercases unquoted aliases, which had produced `undefined` counts.
- **Dashboard third-party count:** fixed `thirdParty.c` on an array → destructured `[{ c: thirdPartyCount }]` ; now returns 7.
- **View Report:** replaced bare `window.open()` (unauthenticated tab → 403) with `fetch` + auth headers → blob URL. Production bundle rebuilt (`main.c2089479.js`).
- **Host sync (.85):** remote host was a Sep 16 build without the DPIA module or `dpia_*` tables. Code + build + PostgreSQL data migrated; service relaunched under PM2.
- **Verified dashboard/report values:** 14 DPIAs · 4 completed / 10 open · 6 high-risk · 13 open risks · 7 third-party · 12 data categories.

9. Build & Test Commands

```
npm start          # CRA dev server (http://localhost:3000)
npm run build       # production bundle → build/
node server.js      # prod web + API (http://localhost:5000)
npm run test:backend # Jest API tests (tests/server.test.js)
npm run test:all     # backend + CRA tests (CI)
```